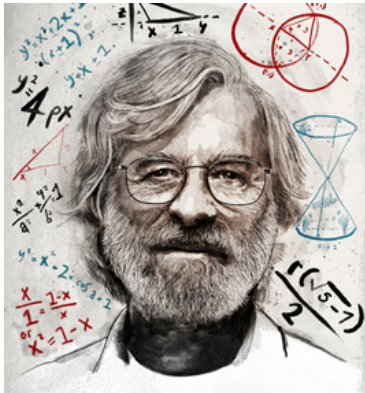




*Leslie Lamport*

„Coding is to Programming,  
what typing is to writing“



Coding needs a **purpose**... we want to write a programm that does **something useful**

//01  
**GLACIERS  
ARE MELTING.**



//02  
**EXTREME EVENTS  
ARE INCREASING**

//03  
**TEMPERATURE  
IS RISING.**



*“Garbage brought disease to our streets.  
We learned to dispose of it.  
Sewage poisoned our waters.  
We learned to treat it.  
CO<sub>2</sub> threatens to change our climate.  
Hence, we must learn how to capture and bury it.”*

W.S. Broecker: Climatic change; are we on the brink of a pronounced global warming? In: Science. v. 189, n. 4201, 1975, S. 460–463

# Programmierprojekt

## The manmade Carbon Cycle



### Voluntary Carbon Markets (VCMs):



- Connect carbon consumers and producers
- Acquiring and burning carbon certificates is used to offset emissions

# Programmierprojekt

## Offsetting fueled by Token

### Definition

A Token is a transferable representation of a right of ownership, access or service, administrated by a distributed ledger.

### Distinctions

1. Native/Protocol Token: Are implemented directly into the blockchain protocol (Bitcoins)
2. Second Layer Token: Are implemented in the programmable memory of a blockchain (e. g. ERC 20)

### Common ways of storing token

#### Transaction History

Alice → Bob (2)  
Charlie → Alice (2)



#### Accountings Table (fungible)

|         |   |
|---------|---|
| Alice   | 2 |
| Bob     | 1 |
| Charlie | 4 |

ERC20



#### Table (non-fungible)

|         |                |
|---------|----------------|
| Alice   | [0x132, 0x232] |
| Bob     | [0x333]        |
| Charlie | [0x123]        |

0x132 ->



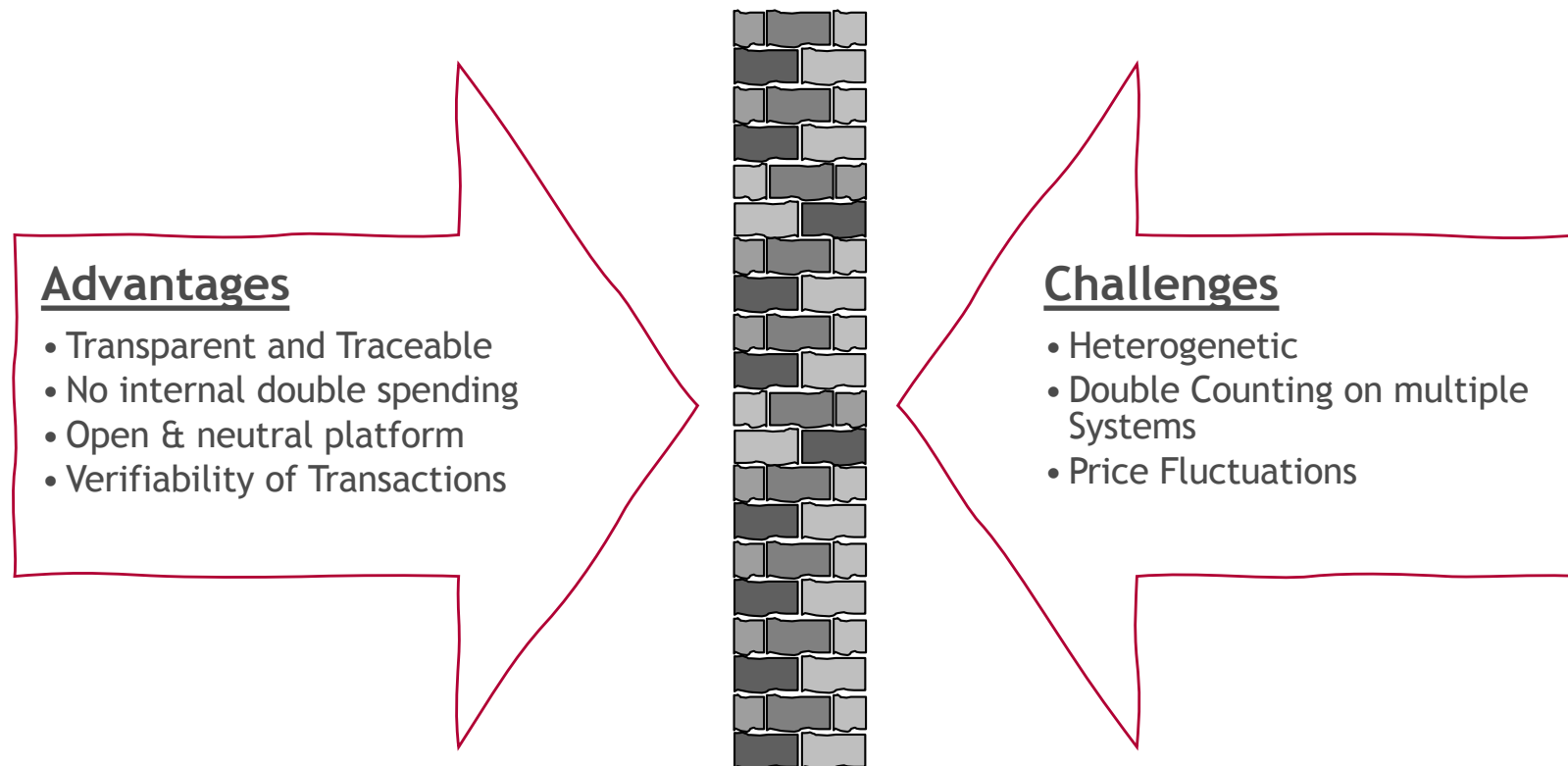
# Programmierprojekt

## What do you think?



**Which measures for decarbonization  
do you know?**

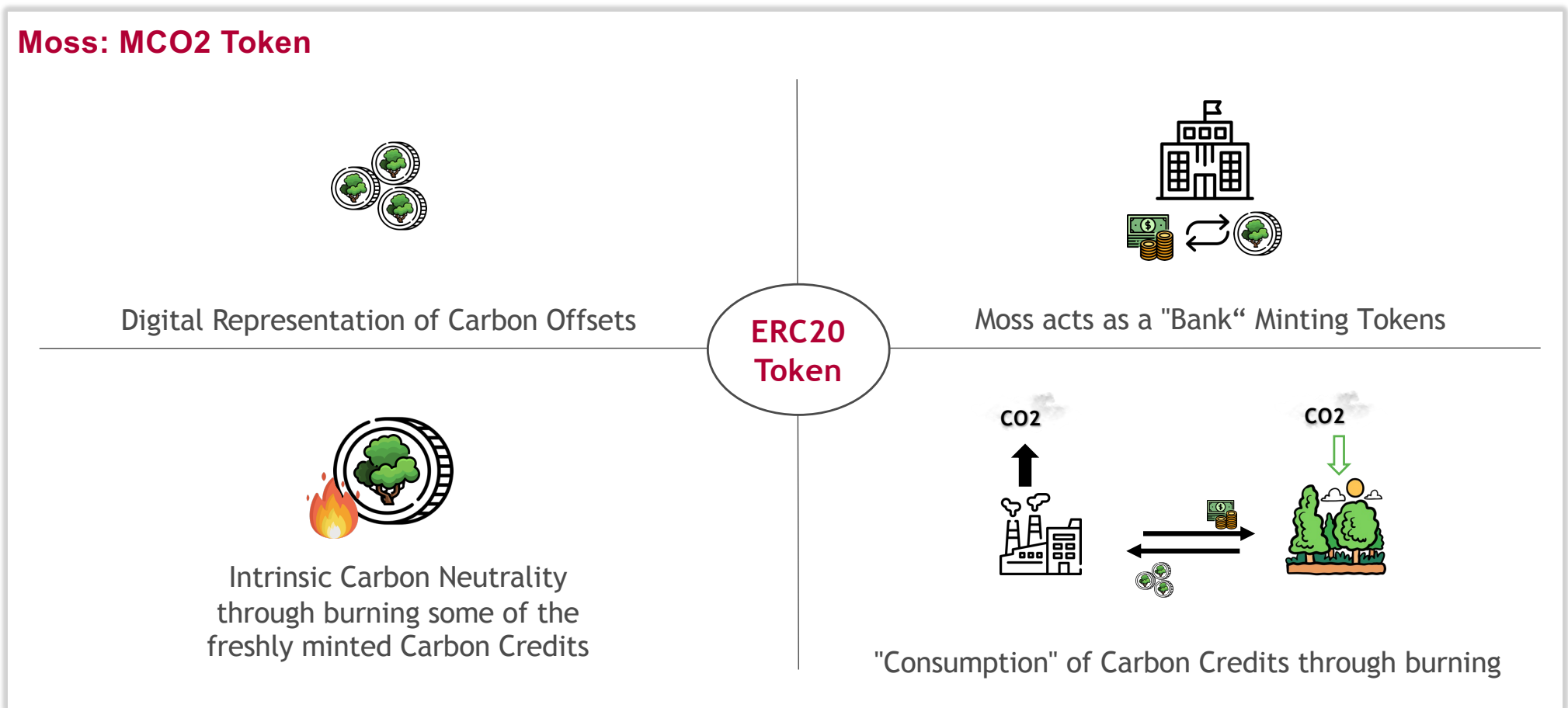
**What are some challenges of current  
solutions?**



**Advantages seem to outweigh challenges**

- Valuations of VCM projects on blockchains are rising
- European Energy Exchange Group develops Token

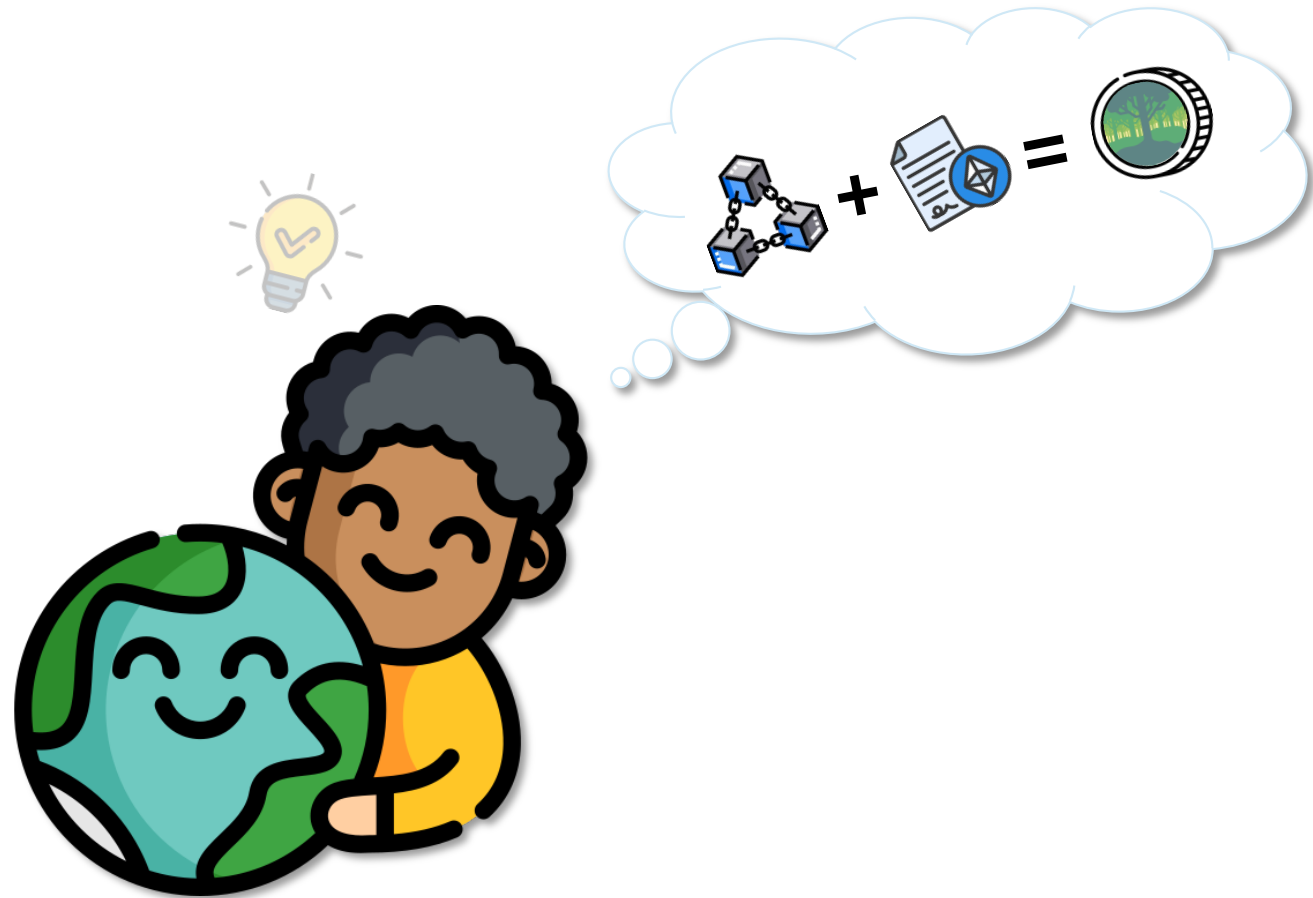
### Moss: MCO2 Token





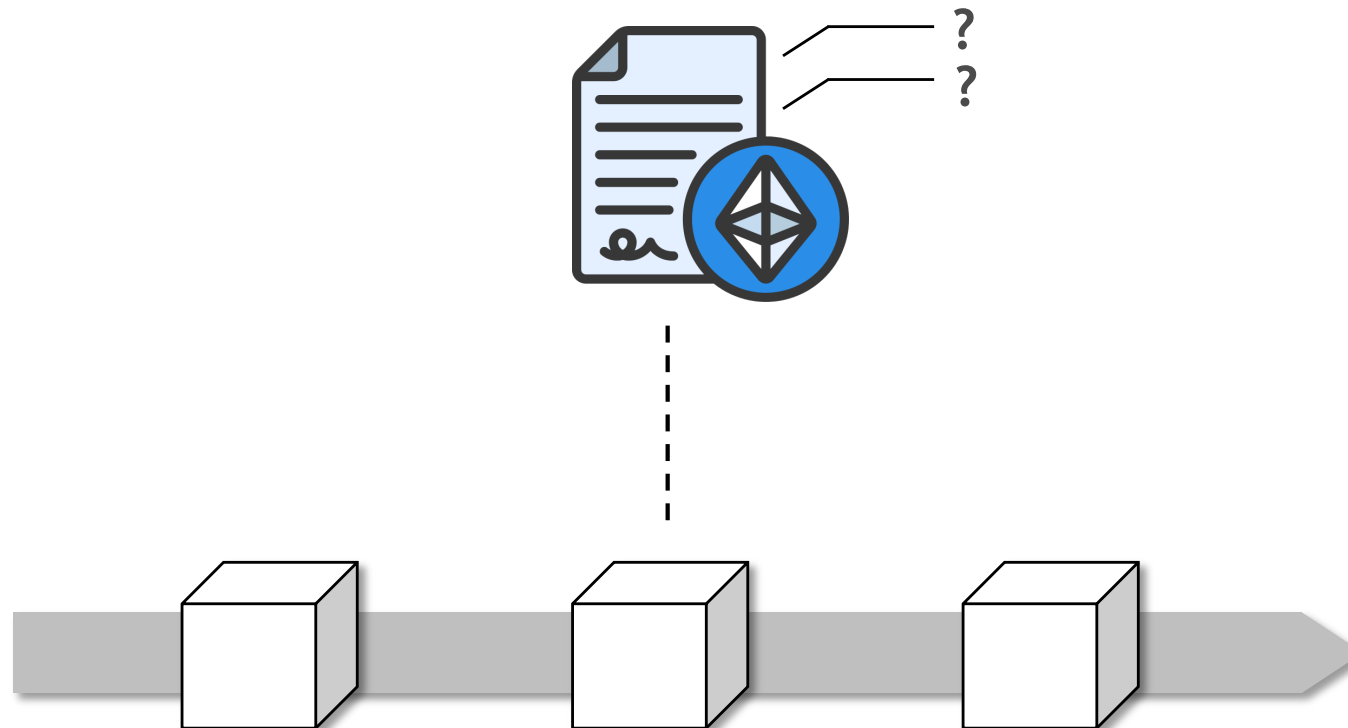
# Programmierprojekt

## Hilf mir den Planeten zu retten!



# Programmierprojekt

## Welche Funktionen werden benötigt?



# Programmierprojekt

## Aufgabe 0: Solide Basis schaffen

*[totalSupply]*

Gibt aus, wie viele  
ERC-20 Token  
existieren

*[Balances]*

Datenstruktur, um  
die Anzahl an Token  
pro Account zu  
verwalten

*burn()*

„Verbrennt“ eine  
Anzahl an Token

*balanceOf()*

Gibt die Anzahl der  
Token auf einer  
Adresse zurück

*transfer()*

Ermöglicht den  
Transfer von Token  
zu einer Adresse

//Funktionen:

```
burn(uint256 _amount) public
balanceOf(address _owner) public view returns (uint256 balance)
transfer(address _to, uint256 _amount) public returns (bool success)
```

from <https://github.com/ethereum/EIPs/blob/master/EIPS/eip-20.md>



Programmiert einen Smart Contract, der diese Funktionen implementiert.  
Der Total Supply soll vom Constructor mit 100 Token für den Owner initiiert werden.

# Programmierprojekt

## Gruppeneinteilung



### Gruppe

Tomas, Mike, Lukas S.

Maximilian, Louisa, Ines

Jonathan, Julia, Anna

Annalena, Janelle, Lorenz

Michael, Paul, Valentin

### Gruppe

Max, Markus, Sven

Sebastian, Antonius, Lukas B.

Simon, Lukas M., Michael

# Programmierprojekt

## Aufgabe 1: Große Fische an Land ziehen



Ich möchte, dass auch große Unternehmen und Institutionen unseren Offset-Token nutzen. Sie werden unserem Offset-Ökosystem mehr Popularität und Nützlichkeit bringen.

Damit das klappt, müsste ich einige Token im Namen meines Unternehmens verwalten.



*Hmmm...*  
Welche Funktion brauche ich?  
Kannst du diese implementieren?



# Programmierprojekt

## Aufgabe 1: Große Fische an Land ziehen

### *[allowancesBurn]*

Datenstruktur, um die Menge an Offset-Token pro Delegate pro Account zu verwalten

### *burnFrom()*

Ermöglicht das burnen durch Dritte abhängig von der allowance

### *addAllowBurn()*

Ermöglicht Dritte zu burnen und setzt allowance fest

//Funktionen:

```
burnFrom(address _from, uint256 _amount) public returns (uint256 remaining)
addAllowBurn(address _burner, uint256 _amount) public returns (uint256 total)
```

from <https://github.com/ethereum/EIPs/blob/master/EIPS/eip-20.md>



Programmiert einen Smart Contract, der diese Funktionen implementiert.

# Programmierprojekt

## Aufgabe 2: Lizenz zum Token-Drucken



Ich würde gerne Teil deines Ökosystems werden. So kann ich durch mein Hobby Bäume zu pflanzen, Offset-Token erzeugen und diese verkaufen :)

Klar doch! Das klingt nach einer guten Idee.



*Ah, ja...!*  
Ich brauche eine Funktion, mit der berechnete Individuen neue Token generieren können!



# Programmierprojekt

## Aufgabe 2: Lizenz zum Token-Drucken

### *[mintList]*

Datenstruktur, um die Accounts zu verwalten, die berechtigt sind neue Token zu Minten

### *allowMinting()*

Berechtigt Accounts zu minten. Sollte nur durch Owner ausführbar sein

### *mint()*

Generiert neue Token. Sollte nur durch Berechtigte ausführbar sein

//Funktionen:

```
allowMinting(address _minter) public  
mint(uint256 _amount) public
```

from <https://github.com/ethereum/EIPs/blob/master/EIPS/eip-20.md>



Programmiert einen Smart Contract, der diese Funktionen implementiert.  
**! Achtung: Achtet auf die Zugriffs-Berechtigungen bei Funktionen !**

# Programmierprojekt

## Aufgabe 3: Der Markt regelt das



Das Projekt sieht doch schon sehr vielversprechend aus ;) Wir würden Token auch gerne auf einem Markt weiterverkaufen können.

Und wir würden die Token auch gerne kaufen können.



*Hmm, ein Offset-Markt also...*  
Was brauchen wir hierfür überhaupt?



# Programmierprojekt

## Aufgabe 3: Der Markt regelt das

*[prices]*

Datenstruktur, um  
den Token-Preis des  
jeweiligen  
Verkäufers zu  
verwalten

*setPrice()*

Setzt den Token-  
Preis für den  
jeweiligen Verkäufer

*buy()*

Ermöglicht das  
kaufen von Token  
gegen Ether. Falls  
der preis = 0, kein  
Kauf möglich.

//Funktionen:

```
setPrice(uint256 _price) public  
buy(address _from) public payable returns (bool success)
```

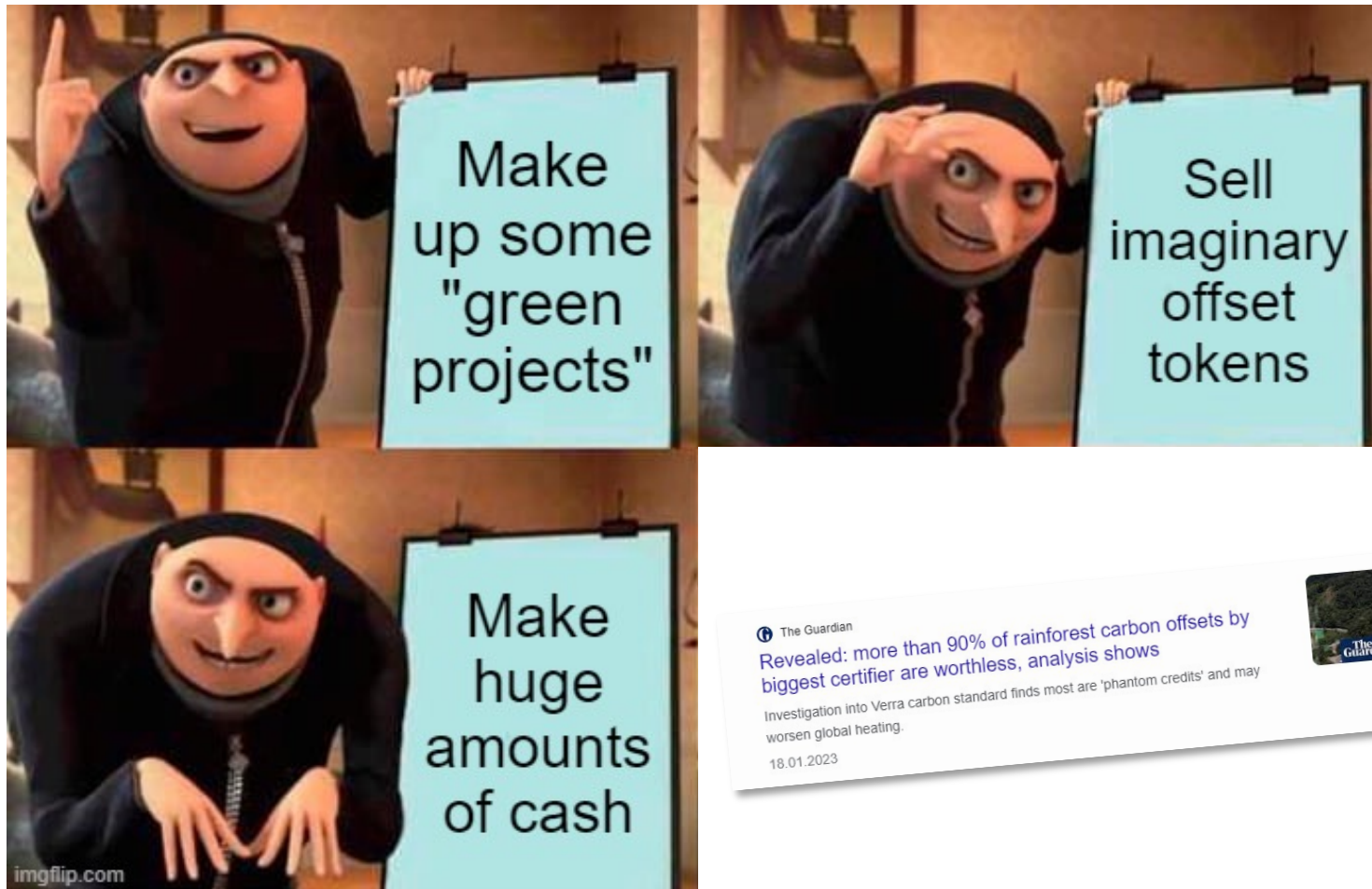
from <https://github.com/ethereum/EIPs/blob/master/EIPS/eip-20.md>



Programmiert einen Smart Contract, der diese Funktionen implementiert.  
**! Achtung: Achtet auf sinnvolle Zugriffsbeschränkungen für „buy()“ !**

# Programmierprojekt

## Abschluss: How to Sell Offsets Online (Fast)



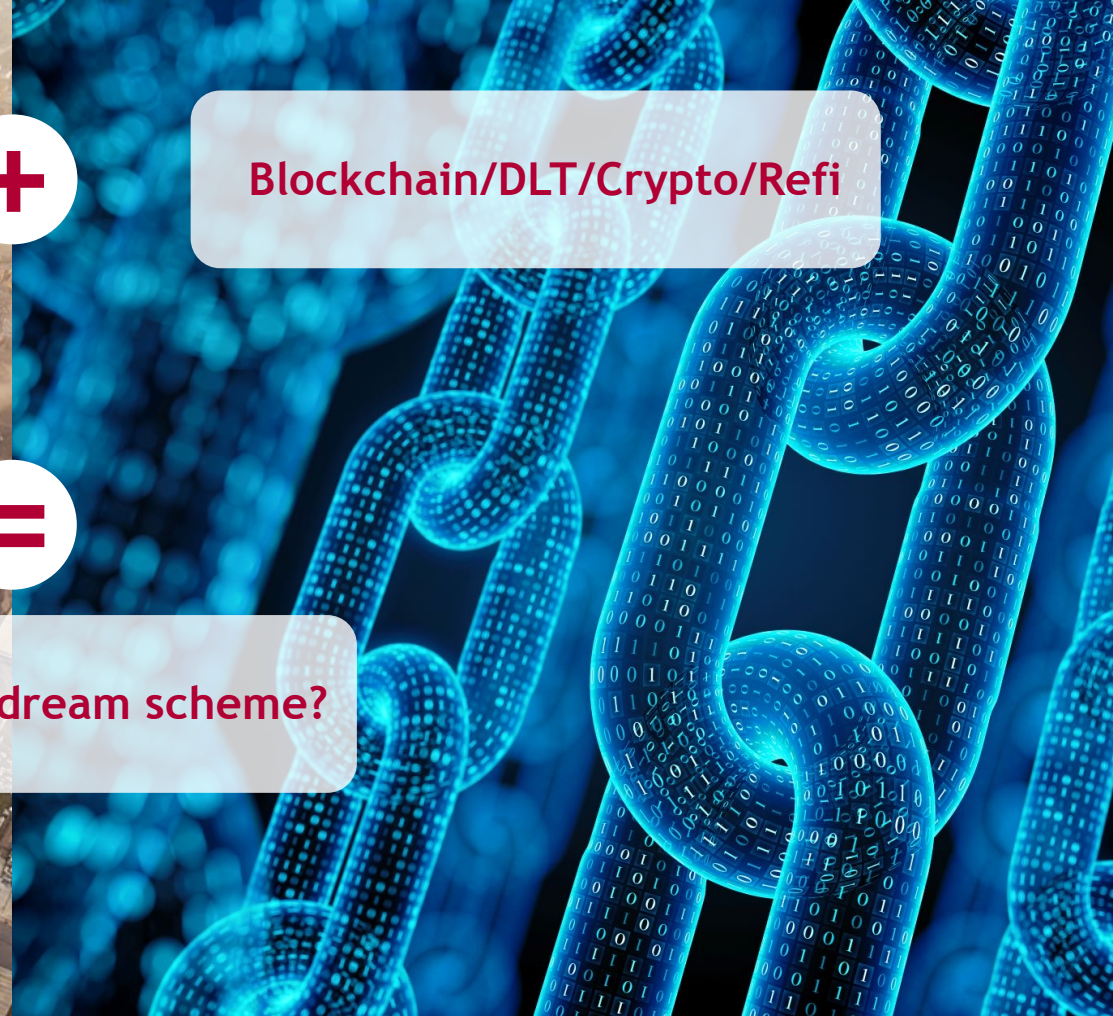
# Programmierprojekt

## Abschluss: How to Sell Offsets Online (Fast)



Carbon offsetting

+



Blockchain/DLT/Crypto/Refi

=

A scammer's dream scheme?

# Vielen Dank für Eure Mitarbeit!

## Wrap-Up und Kontakte:



Prof. Dr. Nils Urbach  
Frankfurt UAS  
[nils.urbach@fim-rc.de](mailto:nils.urbach@fim-rc.de)



Prof. Dr. Jens Strüker  
University of Bayreuth  
[jens.strueker@fim-rc.de](mailto:jens.strueker@fim-rc.de)



Dr. Tobias  
Guggenberger  
University of Bayreuth  
[tobias.guggenberger@fim-rc.de](mailto:tobias.guggenberger@fim-rc.de)



Matthias Babel  
University of Bayreuth  
[matthias.babel@fim-rc.de](mailto:matthias.babel@fim-rc.de)



Marc Principato  
University of Bayreuth  
[marc.principato@fim-rc.de](mailto:marc.principato@fim-rc.de)



Augsburg

Bayreuth

Frankfurt

Luxemburg

München

Stuttgart

